

Apagá el incendio con Tino

Juego interactivo para aprender los tres usos de Tino: Web, Mobile y WhatsApp

Resumen

- Tino real (backend + frontend + Postgres) corriendo en local con Docker, sin internet.
- Pantalla dividida: Tino real a la izquierda, juego pixel art 2.5D a la derecha.
- El visitante usa Tino; el juego observa lo que hace y reacciona (fuegos que se apagan).
- Tres escenarios rotativos de 2 a 3 minutos, hasta 10 acciones cada uno.
- Al final, siempre el dashboard real de Tino con los datos que generó el visitante.
- Simulado solo: datos de demo y el transporte de WhatsApp (no se habla con Meta).

Contenido

- 1. Objetivo y decisiones cerradas
- 2. Arquitectura
- 3. Análisis del grafo y dependencias
- 4. Limpieza del proyecto (qué eliminar, conservar y adaptar)
- 5. Componentes nuevos
- 6. Diseño del juego
- 7. Escenarios
- 8. Pantalla final
- 9. Arte y audio
- 10. Hardware y modo kiosco
- 11. Fases de trabajo
- 12. Riesgos y mitigaciones
- 13. Checklist del día de la feria
- 14. Reglas de trabajo para el proyecto nuevo

1. Objetivo y decisiones cerradas

El objetivo es que los visitantes de la feria **aprendan a usar Tino de manera interactiva** en sus tres usos actuales (Web, Mobile y WhatsApp) a través de un juego simple estilo flash: una persona estresada intenta salvar su empresa en llamas, y cada acción correcta en Tino apaga un fuego.

Tema	Decisión
Quién juega	El visitante, sin presentador. El juego tiene que entenderse solo.
Duración	2 a 3 minutos por partida (límite duro de 150 s).
Escenarios	3 (Web, Mobile, WhatsApp). Un escenario por partida, rotando. El visitante puede seguir con el próximo.
Acciones	Hasta 10 por escenario.
Pantalla	Mitad izquierda: Tino real. Mitad derecha: animación.
Mecánica	Cuando corre el timer de una tarea, el personaje hace esa tarea. Las tareas se califican por prioridad.
Estilo	Pixel art 2.5D (isométrico) muy simple, con todo visible en pantalla.
Ejecución	Local con Docker. No se sube a ningún lado.
Funcionalidad	Se mantiene la funcionalidad real de Tino y todo se maneja desde la aplicación.
Simulación	Solo los datos de demo y el transporte de WhatsApp.
Final	Siempre el dashboard de Tino con las estadísticas de la partida.

2. Arquitectura

```
docker-compose.feria.yml (todo local, sin internet)
├─ db      Postgres 16
├─ backend NestJS real de Tino + módulo nuevo "demo" (solo con DEMO_MODE=true)
└─ frontend Next.js real de Tino + página nueva /feria

/feria
├─ Izquierda: Tino REAL
│   ├── Web      → iframe con /projects/<proyecto-demo>
│   ├── Mobile   → iframe con /mobile dentro de un marco de celular
│   └── WhatsApp → simulador de chat conectado al bot real
├─ Derecha: juego pixel art (Phaser 3)
└─ useDemoState(): consulta GET /demo/state cada ~1 s
    → store del juego (Zustand) → Phaser reacciona
```

- **El juego no modifica Tino: solo lee.** El visitante toca botones reales, los datos cambian en el backend real, y el juego lo detecta (timer activo en la tarea X → el personaje trabaja; tarea X en DONE → se apaga el fuego).
- **Mismo origen:** el iframe y /feria viven en el mismo Next.js, comparten la cookie de sesión y el proxy `app/api/[...path]`. No hace falta configurar CORS ni cookies cruzadas.
- **El asistente funciona sin internet:** `assistant.service.ts` detecta intenciones por reglas y consulta Prisma; no llama a ningún modelo externo.
- **Un solo timer activo a la vez,** igual que en Tino (regla del módulo de tiempo marcada en el grafo).

3. Análisis del grafo y dependencias

El grafo de conocimiento del proyecto (graphify, 3477 nodos, 181 comunidades, sin ciclos de imports) define qué se puede borrar con seguridad.

- Los nodos más conectados del backend (**PermissionUser** 208 conexiones, **PrismaService** 108, **CurrentUser** 107) y el del frontend (**useAuth()** 81) **se conservan:** son el núcleo de Tino real.
- Comunidades que se eliminan: `TrelloImportService`, `trello.adapter.ts`, `IntegrationsController`, `integration-change.service.ts`, `AdminService`, `PlansService`, `WhatsAppDigestService` y las de deploy (Production Pipeline, Google Cloud Run, Vercel, Docker build & publish job).
- Al no haber ciclos de imports, se puede borrar por carpetas y corregir lo que reporte `tsc`.

Dependencias entre módulos del backend (revisadas import por import)

Módulo	Lo usan	¿Se puede borrar?
admin	solo app.module	Sí
plans (módulo)	solo app.module	Sí. Conservar common/plans (lo usan guards y validaciones).
trello-import	solo app.module	Sí
integrations	trello-import, whatsapp-digest.service, whatsapp-digest-format, whatsapp.module	Sí, borrando también el digest de WhatsApp (depende de datos de Trello).
organizations	auth.module, auth.service	No: auth depende de él.
analytics	assistant.service, time-tracking.service	No
assistant	todo el módulo whatsapp	No
users, task-comments	solo app.module	Se conservan: son parte del uso real.

4. Limpieza del proyecto

Criterio: se elimina solo lo que (a) no forma parte de los tres usos (Web, Mobile, WhatsApp) o (b) no funciona sin internet (Google OAuth, Trello, Meta). Todo lo demás queda **igual al original**, para que el visitante aprenda el producto real.

4.1 Copia del proyecto

Copiar sin dependencias, builds, historial de git ni el archivo .env (tiene credenciales reales):

```
robocopy "C:\Users\aaaron\OneDrive\Desktop\Trabajos\Tino\Tino_tasks-dev\Tino_tasks-dev"
"C:\Users\aaaron\OneDrive\Desktop\Trabajos\Tino\Tino_feria"
/E /XD node_modules .next .swc coverage dist .git graphify-out .github docs
.agents "pa subir a git" Resúmenes
/XF .env test-results.json tsconfig.tsbuildinfo tsconfig.build.tsbuildinfo
```

(Es un solo comando; está partido en líneas para que entre en la página.) Después: git init en la carpeta nueva.

4.2 Raíz

Elemento	Acción	Motivo
.github/, docs/, pa subir a git, .agents/	ELIMINAR	Deploy y documentación interna.
docker-compose.yml + docker-compose.local-db.yml	ADAPTAR	Unificar en docker-compose.feria.yml (db + backend + frontend).
.env.example	ADAPTAR	Reemplazar por .env.feria.example.
CLAUDE.md, .claude/	CONSERVAR	Sumar las reglas de trabajo (sección 14).

4.3 Backend

Elemento	Acción	Detalle
modules/admin/, modules/plans/, modules/trello-import/, modules/integrations/	ELIMINAR	Módulos completos.
whatsapp-digest.service.ts, whatsapp-digest-format.ts (+ specs)	ELIMINAR	Quitar su registro en whatsapp.module.ts. Los comandos de "novedades trello" quedan sin efecto.
Flujo de Google OAuth en auth	ELIMINAR	Solo si se aísla sin romper auth.service; si no, queda inactivo sin credenciales.
scripts/provision-pilot-users.ts, PILOT_PROVISIONING.md, pilot-users.example.json	ELIMINAR	Los reemplaza el seed de demo.
coverage/, dist/, Resúmenes/, test-results.json, graphify-out/	ELIMINAR	Artefactos generados.
app.module.ts	ADAPTAR	Quitar los módulos borrados y registrar DemoModule solo si DEMO_MODE=true.
auth, users, organizations, common/active-organization, common/plans	CONSERVAR	Núcleo de autenticación y permisos.
projects, tasks, task-comments, time-tracking, analytics, assistant	CONSERVAR	Funcionalidad real que se enseña.
whatsapp (sin digest)	CONSERVAR	Lógica real del bot.
database, health, reset-db.ts, Dockerfile	CONSERVAR	Infraestructura local.
prisma/schema.prisma	CONSERVAR	Sin tocar. Las tablas de integraciones quedan vacías; cambiar el schema obliga a migraciones.

4.4 Variables de entorno (.env.feria)

- DATABASE_URL apuntando al Postgres del compose.
- Un JWT_SECRET nuevo, con sesiones de larga duración.
- DEMO_MODE=true.
- Sin claves de Meta, Google ni Trello. Si whatsapp.config.ts exige variables, se cargan valores ficticios (el cliente de Meta se reemplaza en modo demo, ver 5.1).

4.5 Frontend: rutas

Ruta	Acción	Motivo
dashboard/, projects/, projects/[id]/	CONSERVAR	Tino Web.
mobile/ (inicio, tareas, detalle, tiempo, resumen, asistente)	CONSERVAR	Tino Mobile.
users/	CONSERVAR	Vista del equipo.
login/	CONSERVAR	Respaldo; en la feria se usa la sesión automática.
api/[...path]/	CONSERVAR	Proxy al backend , imprescindible.
layout.tsx (AuthProvider, TimerProvider)	CONSERVAR	Contextos reales de Tino.
page.tsx (landing)	ADAPTAR	Redirigir a /feria.
admin-panel/, register/, invite/, auth/google/*, auth/setup-password/, workspace/, perfil/, security/	ELIMINAR	Administración, alta de usuarios y OAuth.
integrations/, privacy/, terms/, data-deletion/, manifest.ts	ELIMINAR	No forman parte de los tres usos.

4.6 Frontend: componentes, hooks, lib y tipos

Elemento	Acción	Detalle
components/admin/, marketing/, organizations/, integrations/	ELIMINAR	Fuera de los tres usos.
projects/TrelloImportModal.tsx, mobile/MobileInstallPrompt.tsx	ELIMINAR	Trello y PWA.
hooks: useIntegrations, useIntegrationActivity, useTrelloConnection, useTrelloImport, usePwaInstall	ELIMINAR	
lib: trelloAuthPopup.ts, integrationActivity.ts	ELIMINAR	
types: integration.ts, trello-import.ts, pwa.ts	ELIMINAR	
layout/Navbar.tsx	ADAPTAR	Ocultar links a admin, workspace y perfil.
mobile/dashboard/MobileHomeScreen.tsx	ADAPTAR	Quitar IntegrationActivityBanner.
mobile/MobileProtectedLayout.tsx	ADAPTAR	Quitar MobileInstallPrompt.
app/projects/[id]/page.tsx	ADAPTAR	Quitar ProjectIntegrationPanel y TrelloImportModal.
package.json	ADAPTAR	Agregar phaser y zustand; quitar solo lo que quede sin uso.
Todo lo demás	CONSERVAR	Sin cambios visuales ni de comportamiento. Es la garantía de que se aprende Tino real.

Los adaptados solo pierden referencias a lo eliminado: no cambian su aspecto ni su comportamiento.

4.7 Verificación de la limpieza

- tsc --noEmit sin errores en backend y frontend.

- Backend y frontend compilan, y los tests pasan (eliminando los specs de lo borrado).
- Con docker `compose -f docker-compose.feria.yml up`, Tino funciona en local: login, proyectos, timer, dashboard, mobile y asistente.
- `graphify update .` y luego `graphify query "Trello"` y `graphify query "AdminService"` no devuelven nada, mientras `useAuth` y `PrismaService` siguen presentes.

5. Componentes nuevos

5.1 Backend: modules/demo/ (solo con DEMO_MODE=true)

Endpoint	Qué hace
POST /demo/session	Inicia sesión como el usuario demo y setea la cookie. Nadie tipea contraseñas en la feria.
POST /demo/reset { escenario }	Borra tareas, timers y chat de la organización demo, y carga el seed del escenario. Se llama en cada INTRO.
GET /demo/state	Tareas del proyecto demo (estado y prioridad), timer activo e intenciones consultadas por WhatsApp o el asistente. Una sola consulta para todo el juego.
POST /demo/whatsapp/inbound { text?, selectionId? }	Arma un payload con el formato de Meta y llama a <code>WhatsAppService.handleWebhook()</code> : la lógica real del bot.
GET /demo/whatsapp/messages	Devuelve lo que el bot "envió", para mostrarlo en el chat simulado.

Piezas internas

- **CapturingWhatsAppClient:** reemplaza a `WhatsAppClientService` con la misma interfaz (`sendText`, `sendMenu`, `sendTemplate`). En lugar de llamar a `graph.facebook.com` guarda los mensajes en memoria. El bot no nota la diferencia.
- **Ajustes de modo demo:** `allowQuery()` limita consultas por hora por usuario y en demo todos los visitantes son el mismo usuario, así que se desactiva o se sube; `isDuplicate()` exige IDs únicos de mensaje (los genera el endpoint); la firma de Meta no aplica porque se llama al servicio directamente.
- **DemoSeedService con historial:** empresa "Tino Demo", usuario demo y 4 o 5 compañeros; proyectos con semanas de horas cargadas (para que dashboard, heatmap y respuestas del bot tengan datos); el número de WhatsApp demo ya vinculado; y por escenario, tareas con prioridades, algunas vencidas, algunas sin responsable y compañeros con timers activos.

5.2 Frontend: página /feria

```
src/app/feria/page.tsx
src/game/
├─ flow/GameFlow.tsx          ATTRACT → INTRO → PLAYING → STATS → CONTINUE
├─ flow/rotation.ts           índice de escenario en localStorage (con try/catch)
├─ state/useDemoState.ts      consulta /demo/state y alimenta el store
├─ store/gameStore.ts         fuegos, estrés, tiempo, log, puntaje, historial
├─ rules/fireSystem.ts        tamaño y crecimiento de fuegos según prioridad
├─ rules/scoring.ts           eficiencia de priorización
├─ escenarios/{web,mobile,whatsapp}.ts  fuego ↔ tarea o intención, objetos, animaciones
├─ panels/TinoFrame.tsx       iframe de Tino Web
├─ panels/PhoneFrame.tsx      marco de celular con iframe de /mobile
├─ panels/WhatsAppSimulator.tsx chat estilo WhatsApp: texto, *negritas*, listas del menú
├─ coach/Coach.tsx            guía paso a paso sobre la app real
├─ final/FinalStats.tsx       dashboard real + KPIs del juego
├─ engine/PhaserGame.tsx + scenes/ + objects/
└─ hooks/useIdleReset.ts      20 s sin tocar → modo atracción
```

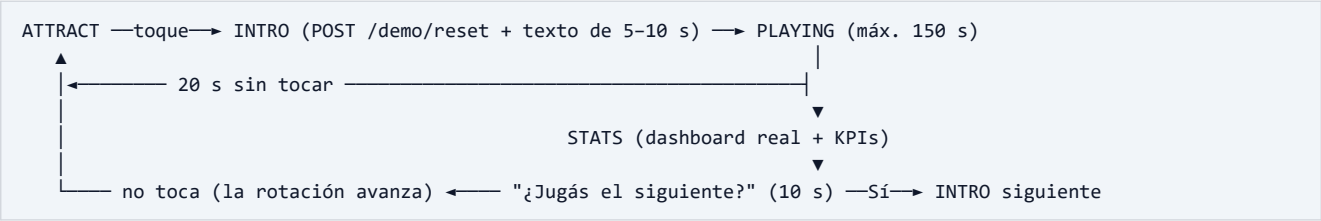
5.3 Coach: el que enseña a usar Tino

- Como el iframe es del mismo origen, /feria puede **resaltar el botón real dentro de Tino** (un recuadro que pulsa sobre "Iniciar timer" de la tarea correcta) sin modificar los componentes de Tino.

- Cada escenario define sus pasos como “elemento a resaltar + texto corto”.
- El coach avanza solo cuando /demo/state confirma que el paso ocurrió.
- **Respaldo:** si un selector cambia, se muestra solo el texto del paso sobre el panel del juego.

6. Diseño del juego

6.1 Flujo de una partida



- La rotación avanza siempre al terminar una partida: el próximo visitante ve otro canal.
- Si un visitante juega los tres escenarios, al final aparece “Tu día con Tino” con el acumulado.

6.2 Cómo el juego lee a Tino

Lo que pasa en Tino (detectado por /demo/state)	Reacción del juego
Timer activo en la tarea X	El personaje va al objeto de X y trabaja en loop.
Timer detenido	El personaje espera: “¿Terminaste? Marcala como hecha”.
La tarea X pasa a DONE	El fuego de X se apaga con humo, baja el estrés y suma puntos.
Timer iniciado en una tarea habiendo otra de mayor prioridad pendiente	La de mayor prioridad crece un nivel, aparece un globo de aviso y queda registrado como error de prioridad.
Consulta por WhatsApp o asistente con la intención correcta	El personaje le muestra el dato a quien lo pidió y el fuego se apaga.
Consulta con la intención incorrecta	“¡Eso no es lo que te pedí!” y el fuego crece un poco.

6.3 Reglas

- **Fuegos:** el tamaño inicial sale del priority real de la tarea (LOW 1, MEDIUM 2, HIGH 3, CRITICAL 4). Las pendientes crecen lento y las CRITICAL más rápido.
- **Estrés:** sube según la suma de fuegos activos y baja de golpe al completar una tarea.
- **Eficiencia de priorización:** porcentaje de veces que el visitante eligió la tarea de mayor prioridad entre las pendientes. Es el KPI estrella de la pantalla final.
- **Fin:** cuando no quedan fuegos o se acaba el tiempo. Nunca hay “perdiste”: siempre se llega a las estadísticas.
- **Guía inicial:** el coach resalta la primera acción; si pasan 8 s sin tocar, vuelve a aparecer.

7. Escenarios

Cada escenario enseña el uso real de su canal, con hasta 10 acciones.

7.1 Escenario 1 — Tino Web, en la oficina (3 tareas, 10 acciones)

Tarea	Prioridad	Objeto y animación
Servidor caído	CRITICAL	Rack con chispas → reparar con llave
Cliente furioso	HIGH	Teléfono sonando → atender
Regar la planta	LOW	Planta mustia → regar

Por cada tarea el visitante: (1) inicia el timer eligiendo la duración en el modal real `StartTimerDurationModal`, (2) lo detiene y (3) la marca como Hecha con las transiciones reales de estado. Son 9 acciones; la décima es **“Abrí el**

Dashboard", y así el visitante llega solo a las estadísticas.

7.2 Escenario 2 — Tino Mobile, en la calle (2 tareas + 1 consulta, ~10 acciones)

Todo dentro de /mobile en el marco de celular:

- Inicio → Tareas → abrir "Entrega atrasada" (CRITICAL) → Iniciar timer.
- Pestaña Tiempo → Detener → Hecha.
- Lo mismo con "Aprobar presupuesto" (HIGH).
- Pedido: "¿Qué quedó atrasado?", que se resuelve con el **Asistente** de la app.
- Cierre en la pestaña **Resumen** real.

7.3 Escenario 3 — WhatsApp, en el aeropuerto (5 pedidos, 10 acciones)

En Tino, WhatsApp es un **canal de consultas**: no inicia timers ni cambia tareas. Por eso aquí los fuegos son pedidos de información de otros personajes, y se apagan consultando lo correcto.

Fuego (pedido de un personaje)	Lo apaga (opción real del menú)
"¡Reunión en 5 minutos y no tenés números!"	Resumen para reunión
"El cliente pregunta qué está atrasado"	Tareas atrasadas
"¿Quién está trabajando ahora?"	Quién trabaja ahora
"Hay tareas que nadie tomó"	Tareas sin responsable
"¿Alguien está sobrecargado?"	Carga del equipo

- Cada pedido se resuelve escribiendo "menú" y eligiendo de la lista real (2 acciones).
- El coach también enseña que se puede **escribir en lenguaje natural** ("¿qué está atrasado?"), porque el bot detecta la intención.
- Prioridad: cada pedido tiene urgencia propia; el primero en arder fuerte es la reunión.

8. Pantalla final (siempre)

Escenario	Qué se muestra
Web	El dashboard real (/dashboard) a pantalla completa.
Mobile	El Resumen real (/mobile/summary) en el celular, más el dashboard web al lado.
WhatsApp	El último mensaje del bot ("Resumen para reunión"), más el dashboard web.

Encima va una franja del juego con KPIs que Tino no calcula: **eficiencia de priorización, fuegos apagados, tiempo de la partida y canal usado**. Como el seed trae historial, los gráficos se ven con datos, y las tareas del visitante aparecen como lo último completado. Después viene "¿Jugás el siguiente?" con cuenta regresiva de 10 s.

9. Arte y audio

9.1 Pixel art 2.5D

Elemento	Especificación
Resolución	Interna de 480×270, escalada x4 a 1920×1080 con pixelArt: true y roundPixels: true.
Proyección	Isométrica 2:1, tiles de 32×16 px. Cada escenario es una habitación de hasta 8×8 tiles, toda visible sin cámara móvil.
Paleta	16 a 24 colores. Fondos azul grisáceo cercanos al #1e3a5f de Tino; fuego naranja y rojo saturado.
Personaje	32×32, 2 direcciones reflejadas: quieto (2 frames), caminar (4), trabajar (4, una variante por tarea), estresado (2), festejar (4).
Fuego	4 tamaños × 4 frames en loop, humo de apagado (5 frames) y chispas para CRITICAL.
Personajes secundarios	Jefe, cliente y compañero que “piden cosas” en el escenario WhatsApp, con globo de diálogo.
Íconos de canal	Monitor, celular o WhatsApp flotando sobre el personaje según el escenario.
Objetos	5 por escenario, en estado normal y en estado “en problema”.
Interfaz del juego	Barra de estrés, reloj y globos de diálogo con fuente pixel (por ejemplo Press Start 2P).
Herramientas	Aseprite, o LibreSprite y Piskel (gratis). Packs CC0 de Kenney.nl. Revisar la licencia de cada pack de itch.io.
Placeholders	Formas simples (fuego = triángulo naranja) hasta la fase de arte.

9.2 Audio

- Música chiptune en loop que se acelera cuando el estrés pasa de 70.
- Efectos: clic, chisporroteo, “fiuuu” al apagar, festejo y alarma del CRITICAL.
- Sistema de sonido de Phaser, sin dependencias extra. Botón de mute para el equipo.

10. Hardware y modo kiosco

- Monitor de 1920×1080 horizontal, idealmente táctil (si no, mouse).
- Chrome en modo --kiosk; sin zoom por gestos (touch-action: manipulation), sin menú contextual ni selección de texto; pantalla sin suspensión.
- **Docker:** levantar el compose antes de abrir el stand. Si algo falla, `docker compose restart`.
- **Sesión:** si expira (401), /feria llama a POST /demo/session y sigue.
- **Respaldo:** segunda notebook con todo levantado y la base cargada.
- **ErrorBoundary** que, ante cualquier error, vuelve al modo atracción.

11. Fases de trabajo

Cada fase cierra con sus tests corriendo y `graphify update` .

Fase	Entregable	Terminado cuando	Tests
0. Copia y limpieza	Sección 4 completa y <code>docker-compose.feria.yml</code>	Tino corre en local sin internet (login, Web, Mobile, asistente); el grafo ya no tiene Trello, admin ni plans	Suites de backend y frontend en verde
1. Modo demo (backend)	DemoModule: sesión, seed de 3 escenarios con historial, reset, state, WhatsApp con captura	Reset en menos de 2 s; el bot responde las 9 consultas desde el endpoint	Seed, reset, state, CapturingWhatsAppClient, inbound → respuesta

Fase	Entregable	Terminado cuando	Tests
2. Esqueleto de /feria	Pantalla dividida, iframe Web, useDemoState, store, reglas, Phaser con placeholders	El escenario Web se juega de punta a punta usando Tino real	fireSystem, scoring, GameFlow, mapeo estado → fuego
3. Mobile y WhatsApp	PhoneFrame, WhatsAppSimulator y rotación	Los 3 escenarios jugables	Simulador (listas y texto), rotación
4. Pantalla final	Dashboard real con KPIs del juego, "¿Jugás el siguiente?" y acumulado	Siempre aparece, con datos de la partida	Cálculo de KPIs y transiciones
5. Arte	Sprites, tiles, 3 escenas y personajes secundarios	Se lee a 3 m de distancia	Carga de assets
6. Coach, audio y kiosco	Resaltado dentro de Tino, sonidos, modo atracción, reinicio por inactividad, sesión automática	30 partidas seguidas sin intervención	Pasos del coach, useIdleReset
7. Ensayo	Pruebas con 5 a 10 personas ajenas al proyecto	Aprenden el flujo sin ayuda en menos de 3 min	Suite completa

La fase 2 ya permite validar la idea. El arte (fase 5) puede arrancar en paralelo apenas estén definidos los escenarios.

12. Riesgos y mitigaciones

Riesgo	Mitigación
Borrar un módulo rompe auth o WhatsApp	Seguir el mapa de dependencias (sección 3): se conservan organizations, analytics, assistant y common/plans.
El dashboard se ve vacío	Seed con semanas de historial.
El límite de consultas de WhatsApp bloquea la demo	Desactivarlo en modo demo (fase 1).
Un cambio en Tino rompe el coach	Selectores estables y respaldo de solo texto.
Docker o la base fallan en la feria	Levantar antes de abrir, restart documentado y notebook de respaldo.
El visitante tarda demasiado en el flujo real	Máximo 10 acciones, coach y límite de 150 s.
El arte se atrasa	Placeholders hasta la fase 5: el juego tiene que funcionar con formas simples.

13. Checklist del día de la feria

- ☐ docker compose up en verde y /feria abre con la sesión demo
- ☐ Un reset y una partida completa de cada escenario
- ☐ Chrome en kiosco, volumen ajustado, pantalla sin suspensión
- ☐ Notebook de respaldo levantada
- ☐ Rotación en el escenario 1
- ☐ QR o folleto para quienes quieran saber más

14. Reglas de trabajo para el proyecto nuevo

Copiar estas reglas al CLAUDE.md de la carpeta de la feria:

- Comentarios concisos solo arriba de cada función; nada en el medio del código.
- Al cerrar cada fase: escribir tests, correrlos y ejecutar `graphify update`.
- Para preguntas sobre el código, consultar primero el grafo (`graphify query / path / explain`).